PROJECT 2

Object Detection in Satellite Images with YOLO

São Paulo edition — PUC-SP region (Perdizes and surrounding areas)
End-to-end project — programmatic collection, annotation, training, evaluation, and inference

1. Objective

The objective of this project is to experience, in practice, the complete lifecycle of a Computer Vision system based on deep learning, from raw data to a trained model. The group will be responsible for all stages: defining the target object, collecting real images, annotating bounding boxes, preprocessing the dataset, training a detection network, and evaluating the results on unseen images.

The central pedagogical message is: about 80% of the effort in an AI project lies in the data, not in the architecture. The model (YOLOv8/YOLOv11) will be practically the same for all groups — what differentiates the results is the quality of the dataset that is built.

In this edition, the geographic scope is the city of São Paulo, with a focus on neighborhoods near the PUC-SP campus in Perdizes. The groups will use as their main image source the public ESRI World Imagery XYZ tiles service, which provides submetric coverage of São Paulo (up to approximately 0.27 m/pixel) with a permissive license for educational use (with attribution). Support notebooks ready for Colab will be provided by the professor.

Item	Description
Course	Machine Learning
Assessment	P2 — Final Project
Modality	Group of up to 5 students
Framework	Ultralytics YOLOv8 / YOLOv11 (PyTorch)
Training environment	Google Colab Free (T4 GPU)
Image source	ESRI World Imagery (public XYZ tiles) + Google Earth Web (supplementary)
Geographic scope	Perdizes, Higienópolis, Pacaembu, Sumaré, Pinheiros, Itaim, and Av. Paulista
Support notebooks	`Projeto_P2_Mosaico_Perdizes.ipynb` (z=18) and `Projeto_P2_Mosaico_Perdizes_HIRES.ipynb` (z=19)
Submission	Annotated dataset + Notebook (.ipynb) + Weights (.pt) + Report + Presentation

2. Dataset — Collection and Target Definition

Each group must build its own dataset from aerial images/orthophotos of the city of São Paulo. NO ready-made dataset will be used. The images will be saved as `.jpg` or `.png` — we will not work with `.tif`/`.tiff` sensor files at this stage.

2.1. Choosing the target object (suggestions for São Paulo)

The group must choose ONE single target object (a single class). The targets below were selected because they are abundant in the PUC-SP region, have a clear geometric shape in top view, and offer interesting pedagogical challenges:

- **Helipads on rooftops (RECOMMENDED).** São Paulo is the city with the largest urban helicopter fleet in the world. The white H inside a circle is an almost unique visual pattern of the city. Collection is concentrated around Av. Paulista, Faria Lima, Berrini, Itaim, Vila Olímpia, and Brooklin (all within 6 km of PUC-SP).
- **Pools on rooftops and in backyards.** Abundant in Perdizes, Higienópolis, Pacaembu, Sumaré, and Pinheiros. Real-world case: Brazil's Federal Revenue Service has already used pool detection in aerial photos to cross-check with property tax and income tax declarations.
- **Clay tennis courts.** Distinctive orange color, standardized shape. Clubs in the west zone (Paulistano, Pinheiros, Tietê, Hebraica) and rooftops.
- **Five-a-side soccer fields (synthetic grass).** Uniform green that stands out from the surroundings. Common in the west zone and along Marginal Pinheiros.
- **Gas stations.** White rectangular canopy over the pumps, standard layout, spread throughout the city.

Avoid ambiguous objects (e.g., trees, ordinary houses, cars) — they generate many false positives and discourage the group. It is allowed to choose a target different from the suggestions above, as long as it is approved by the professor before collection begins.

2.2. Programmatic collection via ESRI World Imagery (recommended)

Image collection will be done through the public ESRI World Imagery XYZ tiles service (`server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer`). Decisive advantages over manual screenshots: submetric coverage for São Paulo (0.55 m/pixel at z=18 and 0.27 m/pixel at z=19, the practical maximum for this region), permissive license for educational use with attribution, and — above all — the ability to scan an entire neighborhood in seconds, generating a mosaic of standardized tiles (256×256 px each).

Note about GeoSampa. The São Paulo city government provides ultra-high-resolution orthophotos (10 cm/px in urban areas) at `geosampa.prefeitura.sp.gov.br`, with a permissive license and no registration required. Access, however, is via manual download on the portal (there is no public API), which is outside the scope of the base project. Groups interested in using this source as a differentiator may consult the separate document `GeoSampa_passo_a_passo.md`.

Expected procedure (support: notebooks `Projeto_P2_Mosaico_Perdizes.ipynb` and `Projeto_P2_Mosaico_Perdizes_HIRES.ipynb`):

- Choose the zoom according to the target: `z=18` (~0.55 m/px) for pools, courts, and fields; `z=19` (~0.27 m/px) for helipads and smaller targets.
- Define the bounding box of the neighborhood(s) to scan, in the format `(lon_min, lat_min, lon_max, lat_max)`. Example: Perdizes ≈ `(-46.685, -23.545, -46.665, -23.530)`; Itaim ≈ `(-46.685, -23.590, -46.665, -23.575)`; Paulista ≈ `(-46.660, -23.572, -46.640, -23.555)`.
- Run the script below (or the support notebooks with ready outputs) — it converts the bounding box into XYZ tiles and saves each one as `tile_z{z}_x{x}_y{y}.jpg`.

```
import math, os, requests
from concurrent.futures import ThreadPoolExecutor

URL = ('https://server.arcgisonline.com/ArcGIS/rest/services/'
       'World_Imagery/MapServer/tile/{z}/{y}/{x}')
HEADERS = {'User-Agent': 'PUC-SP-AM-Aula-Educacional/1.0'}
BBOX = (-46.685, -23.545, -46.665, -23.530)  # Perdizes
ZOOM = 18  # use 19 if the target is a helipad
OUT = 'mosaico_perdizes'; os.makedirs(OUT, exist_ok=True)

def deg2tile(lat, lon, z):
    n = 2.0 ** z
    x = int((lon + 180.0) / 360.0 * n)
    lat_rad = math.radians(lat)
    y = int((1.0 - math.asinh(math.tan(lat_rad)) / math.pi) / 2.0 * n)
    return x, y

lon0, lat0, lon1, lat1 = BBOX
x_min, y_max = deg2tile(lat0, lon0, ZOOM)
x_max, y_min = deg2tile(lat1, lon1, ZOOM)

def baixar(args):
    z, x, y = args
    p = f'{OUT}/tile_z{z}_x{x}_y{y}.jpg'
    if os.path.exists(p):
        return
    r = requests.get(URL.format(z=z, x=x, y=y), headers=HEADERS, timeout=20)
    if r.status_code == 200 and len(r.content) > 2521:  # filters placeholder
        open(p, 'wb').write(r.content)

jobs = [(ZOOM, x, y) for x in range(x_min, x_max + 1) for y in range(y_min, y_max + 1
with ThreadPoolExecutor(max_workers=4) as ex:
    list(ex.map(baixar, jobs))
print(f'{len(jobs)} tiles in {OUT}/')
```

- Mandatory attribution: include “Source: Esri, Maxar, Earthstar Geographics, and the GIS User Community” in any figure, slide, or report that reproduces the tiles or the mosaic.
- Mandatory manual curation: after generating the mosaic, each team member must review the tiles and discard those that do not contain target objects (e.g., tiles covering only

rooftops without helipads, parks, roads). This curation is part of the evaluation.

- Each neighborhood bounding box must be documented in a spreadsheet (neighborhood, zoom used, responsible member, number of generated tiles, number of approved tiles).

2.3. Manual collection via Google Earth Web (supplementary)

Manual collection via Google Earth Web is reserved for isolated targets that the student identifies while navigating the city and wants to incorporate into the dataset. DO NOT use it for volume — Google's terms restrict bulk screenshots and the IP is blocked after a few hundred requests.

- Keep the same zoom level in all captures (consistent scale).
- Capture approximately square areas and resize them to 640×640 pixels.

2.4. Dataset volume and split of work

- Minimum volume: 200 images with the target object per group (after mosaic curation).
- Geographic diversity: at least 3 different neighborhoods in the training set.
- Each team member annotates their own share — one single member is not allowed to annotate everything.
- Reserve at least 1 entire neighborhood outside the training data for the final generalization test.

3. Technical Requirements

3.1. Image annotation (labeling)

- Mandatory tool: Roboflow (recommended) or CVAT.ai — both run 100% in the browser.
- Draw tight bounding boxes around each instance of the object.
- Define a written annotation standard within the group (e.g., what to do when the object is cut off by the border, partially covered by shadow, or reflecting sunlight).
- All team members must annotate — one single member is not allowed to annotate everything.

3.2. Preprocessing and splits

- Resize all images to 640×640 (configure this in the annotation tool itself).
- Data augmentation in Roboflow (generate additional versions without labeling again): 90° rotation, horizontal and vertical flip, small brightness/contrast variations.
- Split: 70% train / 20% validation / 10% test (configurable in Roboflow).
- Export in YOLOv8 format (compatible with YOLOv11) and use the Roboflow API key in Colab to download the dataset.

3.3. Training in Google Colab (T4 GPU)

Use the `ultralytics` library and the YOLOv8n model (Nano, ~6 MB) or YOLOv11n. These models were chosen because they fit comfortably in the free Colab T4 GPU and train quickly for small datasets.

```
!pip install -q ultralytics roboflow

from ultralytics import YOLO

model = YOLO('yolov8n.pt')  # or 'yolo11n.pt'
results = model.train(
    data='data.yaml',       # generated by Roboflow
    epochs=30,              # 30-50 epochs are enough
    imgsz=640,
    batch=16,               # safe on Colab Free (reduce to 8 if memory is insufficie
    device=0,               # GPU
    seed=42,                # reproducibility
    project='runs', name='exp1'
)
```

- Fix a seed and record all hyperparameters used.
- Save the best checkpoint (the framework already generates `best.pt` automatically in `runs/detect/exp1/weights/`).
- Run at least 2 training rounds varying 1 hyperparameter (e.g., `epochs`, `batch`, or `imgsz`) and compare them.

3.4. Evaluation

- Mandatory metrics: mAP@0.5, mAP@0.5:0.95, precision (P), and recall (R) on the test set.
- Confusion matrix (generated automatically by Ultralytics).
- Loss curves (`box`, `cls`, `dfl`) and P/R curves per epoch.
- Qualitative analysis: show 5 clear correct detections, 3 false positives, and 3 false negatives with a plausible explanation for each error.

3.5. Inference on unseen images

Each group must generate a mosaic of an entire neighborhood NOT used in training and run the model over all tiles. At least 5 of these tiles must be discussed in the report (correct detections, false positives, false negatives). This is the stage where it is verified whether the model truly generalizes to a new region of São Paulo.

```
from ultralytics import YOLO

model = YOLO('runs/detect/exp1/weights/best.pt')
results = model.predict(
    source='imagens_ineditas/',
    save=True,
```

```
    conf=0.25
)
```

3.6. Web application (OPTIONAL — extra credit)

As a differentiator, the group may build a simple application in Streamlit or Gradio that allows the user to upload a satellite image and visualize the detections from the trained model. It is worth up to 1.0 extra point (the maximum grade remains 10.0).

- Load `best.pt` and expose an upload endpoint.
- Show the image with the drawn boxes and the confidence of each detection.

4. Deliverables

- Roboflow project link (or zipped annotated dataset folder) with `train / valid / test`.
- Jupyter Notebook (`.ipynb`) running end-to-end in Colab, with cells for dataset download, training, evaluation, and inference.
- `best.pt` file of the best trained model.
- `runs/` folder with logs, curves, and confusion matrix generated by Ultralytics.
- `imagens_ineditas/` folder with the 5 new images and inference results.
- PDF report (5 to 8 pages): introduction, target definition, collection and annotation process, methodology, results, error analysis, limitations, and conclusion.
- Slide presentation (PDF or PPTX) of up to 10 minutes.
- (Optional) Code and execution instructions for the Streamlit/Gradio app.

5. Evaluation Criteria

The final project grade (0 to 10) will be composed of the following criteria. Note: the maximum grade (including extra credit) is limited to 10.0 points.

#	Criterion	Points
1	Programmatic collection via ESRI XYZ (script, documented bounding box and zoom, mosaic curation, correct attribution)	1.5
2	Annotation quality (tight boxes, consistency among team members)	2.0
3	Preprocessing and augmentation (resize, train/val/test splits)	1.0
4	Training and monitoring (curves, hyperparameters, reproducibility)	1.5
5	Evaluation (mAP@0.5, mAP@0.5:0.95, confusion matrix, error analysis)	1.5
6	Inference on an unseen São Paulo neighborhood (mosaic outside training)	0.5
7	Technical report (clarity, justifications, discussion of limitations)	1.0
8	Oral presentation and group command of the content	1.0
—	**TOTAL**	**10.0**

#	Criterion	Points
+	Web application in Streamlit/Gradio (OPTIONAL — extra credit)	+1.0

6. Rules and Notes

- Groups of 3 to 5 members. Individual work only with prior professor approval.
- All team members must be able to explain any part of the work during the presentation (annotation, training, metrics, inference).
- It is allowed to consult the official documentation of Ultralytics, Roboflow, and tutorials, as long as they are cited in the report.
- Code copied without understanding and without citation will be considered plagiarism and will result in a zero grade.
- Use of generative AI is allowed as a support tool, but it must be declared in the report (which tool, at which stages, and what was effectively used).
- Images must be from public areas (satellite view). Do not annotate people, vehicle license plates, or any personally identifiable data.
- Work submitted after the deadline will lose 1.0 point per day of delay.

7. Final Tips

- Start collection and annotation immediately — this is the part that consumes the most time.
- Agree as a group on a written annotation standard BEFORE starting labeling, to avoid inconsistencies among team members.
- Use `batch=16` as a starting point; if a CUDA out of memory error appears, reduce it to 8.
- In Colab Free, save the dataset and weights to Google Drive — the session expires and anything stored only in `/content` is lost.
- Before submitting, run the notebook from scratch (`Runtime → Restart & run all`) to guarantee reproducibility.
- Compare at least 2 experiments in the report (e.g., 30 vs. 50 epochs, or `batch=8` vs. `16`).

8. Group Identification

Fill in the data below on the first page of the notebook and the report:

Good work!

#	Full Name	Student ID
1		
2		
3		

#	Full Name	Student ID
4		
5		